CEBU INSTITUTE OF TECHNOLOGY
UNIVERSITY

IT342-[G3]
SYSTEMS INTEGRATION AND ARCHITECTURE 1

FUNCTIONAL REQUIREMENTS SPECIFICATION (FRS)

Project Title: Patch Notes

Prepared By: Abregana, Justin Patrick A.

Date of Submission: 14/02/2026

Version: 1

Table of Contents

1. 3
   1.1. 3
   1.2. 3
   1.3. 3
2. 3
   2.1. 3
   2.2. 3
   2.3. 3
   2.4. 4
3. 4
   3.1. 4
   3.2. 4
4. 5
5. 5
   5.1. 5
   5.2. 6
   5.3. 7
   5.4. 8
   5.5. 9
6. 10

1. Introduction

1.1. Purpose

The Patch Notes system is designed to serve as a lightweight gaming social platform where gamers can share posts about their gaming experiences and communicate with other users through direct messaging. This document is intended for developers and stakeholders involved in the design and implementation of the system.

1.2. Scope

The system will provide user registration and authentication, post creation and viewing, a dashboard displaying recent posts, direct messaging between users, and user profile management. It excludes voice/video chat, push notifications, social media login, advanced content moderation, end-to-end encryption, native mobile applications, and analytics dashboards.

1.3. Definitions, Acronyms, and Abbreviations

- **JWT** — JSON Web Token, used for token-based authentication
- **API** — Application Programming Interface
- **REST** — Representational State Transfer
- **SDD** — System Design Document
- **FRS** — Functional Requirements Specification
- **CRUD** — Create, Read, Update, Delete
- **BCrypt** — A password hashing function used for secure password storage
- **Spring Boot** — A Java-based backend framework
- **React** — A JavaScript frontend library
- **Kotlin** — Programming language used for Android mobile development

2. Overall Description

2.1. System Perspective

Patch Notes is a three-tier web and mobile application. The Spring Boot backend serves as the API layer, the React web application provides the desktop/browser interface, and the Android Kotlin app provides the mobile interface. All three communicate through RESTful API calls to the same backend at http://localhost:8080/api.

2.2. User Classes and Characteristics

- **Gamers** — Primary users who register, create posts, view content from others, and send direct messages within the gaming community
- **Community Members** — Users who interact with posts and engage in conversations with other platform members

2.3. Operating Environment

- **Backend:** Java 17, Spring Boot 3.x, Spring Security, Spring Data JPA, Maven
- **Database:** MySQL

- **Web Frontend:** React 18, Vite, Axios
- **Mobile:** Kotlin, Android Studio, Retrofit
- **Browsers Supported:** Google Chrome, Mozilla Firefox, Microsoft Edge (latest versions)
- **Screen Sizes:** Mobile 360px+, Tablet 768px+, Desktop 1024px+
- **Operating Systems:** Windows, macOS, Linux

2.4. Assumptions and Dependencies

- MySQL database must be installed and running locally
- Backend must be running on port 8080 before the web or mobile app can function
- Android emulator uses 10.0.2.2 to connect to the local backend
- Users must have a valid internet/network connection
- JWT tokens are stored client-side (localStorage for web, SharedPreferences for mobile)

3. System Features and Functional Requirements

Describe each major feature of the system and its functional requirements.

3.1. Feature 1: User Authentication

Description: Allows users to register and log in to the platform securely.
Functional Requirements:

- Users must be able to register with email, password, username, first name, last name, age, and gender
- Passwords must be hashed using BCrypt before storage
- Users must be able to log in with email and password
- A JWT token must be returned upon successful login or registration
- Duplicate email registration must be prevented
- Users must be able to log out

3.2. Feature 2: Post Management

Description: Allows authenticated users to create and view gaming-related posts on the dashboard feed.
Functional Requirements:

- Authenticated users must be able to create a new post with text content
- All posts must be visible on the main dashboard feed
- Posts must be retrieved via GET /posts
- Posts must be created via POST /posts

3.3. Feature 3: Direct Messaging

Description: Allows authenticated users to send and receive messages with other users.
Functional Requirements:

- Authenticated users must be able to select another user and send a message
- Messages must be stored and displayed in a conversation thread

- Messages must persist across sessions

3.4. Feature 4: User Profile Management

Description: Allows authenticated users to view and update their profile information.
Functional Requirements:

- Authenticated users must be able to view their profile page
- Users must be able to update their username and bio
- Profile changes must be saved and reflected immediately
- Profile must be accessible via GET /profile
- Profile updates must be submitted via PUT /profile

3.5. Feature 5: Dashboard

Description: Central hub displaying recent posts and user information after login.
Functional Requirements:

- Authenticated users must be redirected to the dashboard after login
- The dashboard must display recent posts from all users
- The dashboard must display the logged-in user's username and email
- Protected pages must not be accessible when logged out

4. Non-Functional Requirements

Specify system quality attributes such as performance, security, usability, reliability, etc.

- **Performance:** API response time must be ≤ 2 seconds; page load time ≤ 3 seconds; database queries within 500ms; supports 50–100 concurrent users
- **Security:** HTTPS for all communication; JWT-based authentication; BCrypt password hashing; input validation to prevent SQL injection; XSS protection
- **Usability:** New users should be able to register and create a post within 5 minutes; UI must be simple and consistent; error messages must be clear and actionable
- **Compatibility:** Must work on Chrome, Firefox, and Edge; responsive across mobile, tablet, and desktop screen sizes; compatible with Windows, macOS, and Linux

5. System Models (Diagrams)

Insert the necessary diagrams for the system:

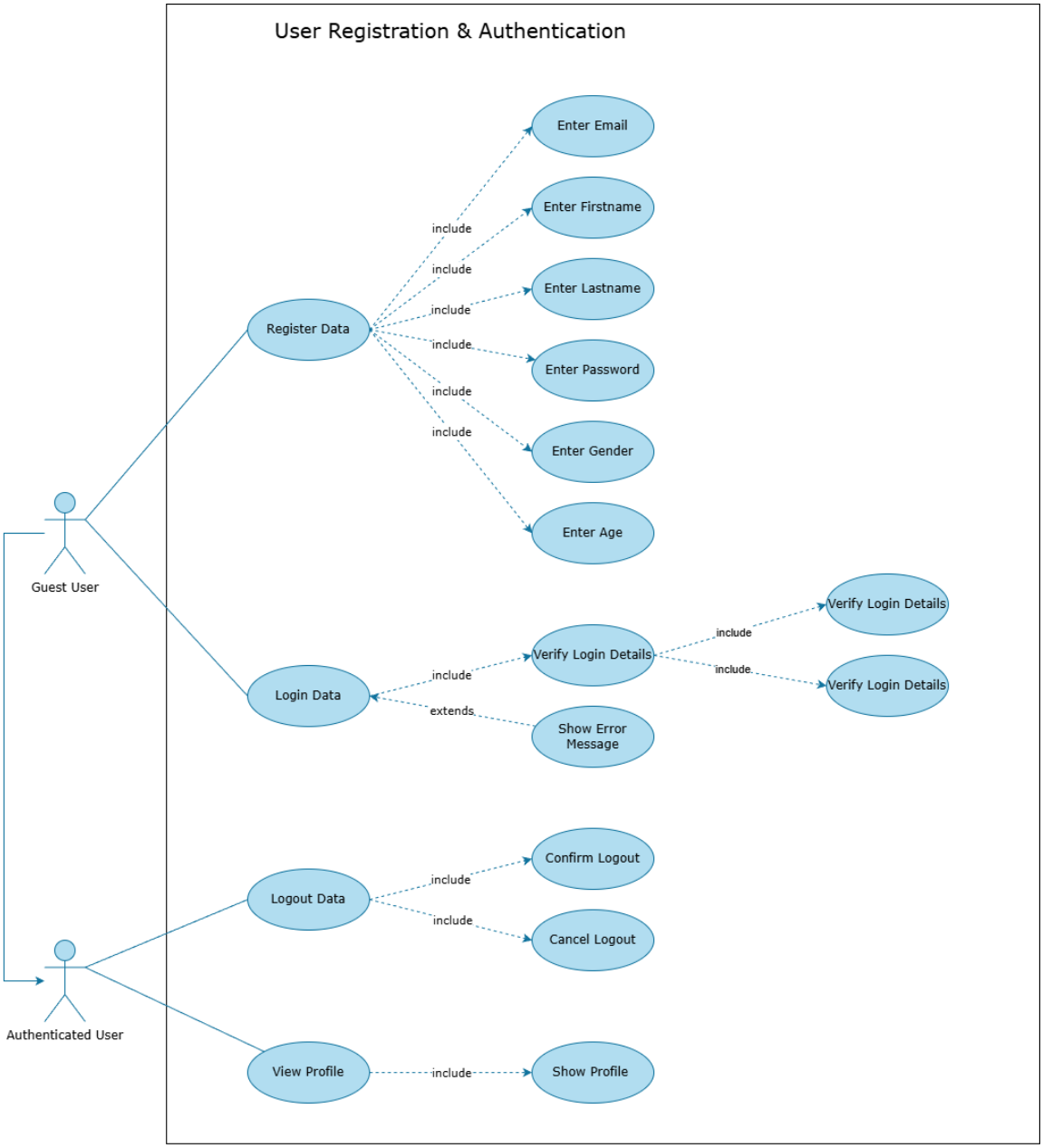
5.1. ERD

Insert ERD here

Table	
PK	**<u>userID</u>**
	email
	firstname
	lastname
	password
	age
	gender

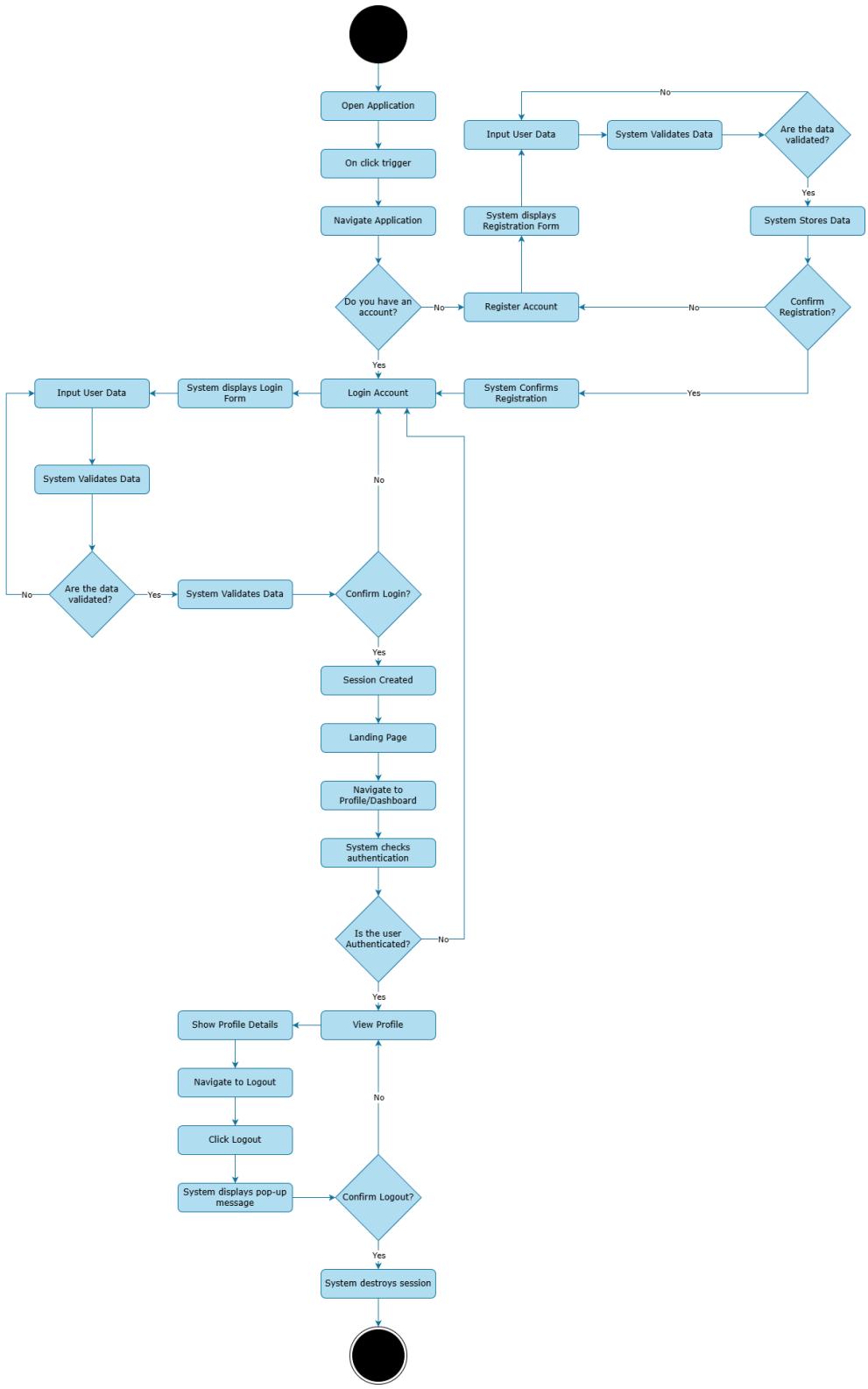
5.2. Use Case Diagram

Insert ERD here



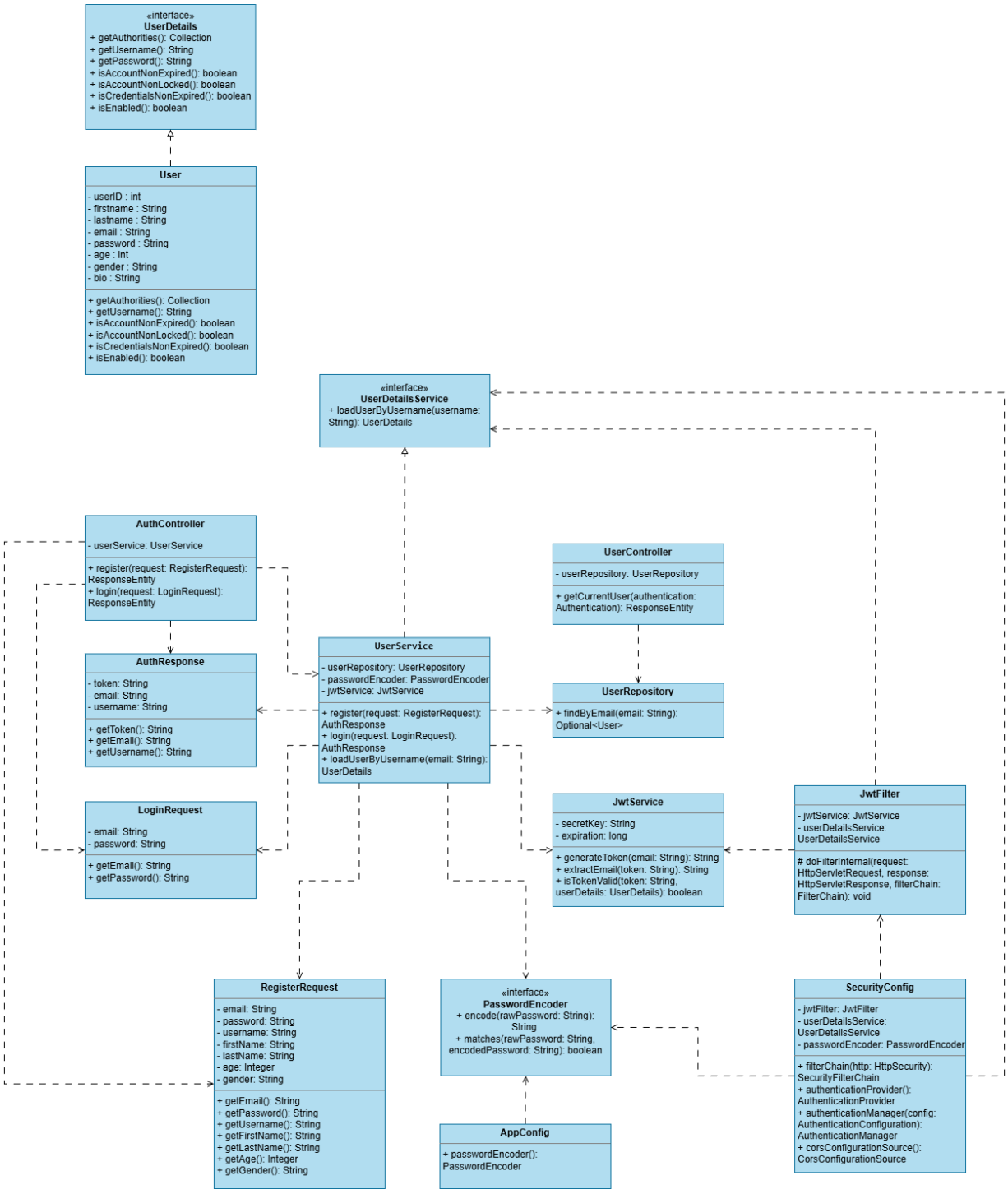
5.3. Activity Diagram

Insert ERD here



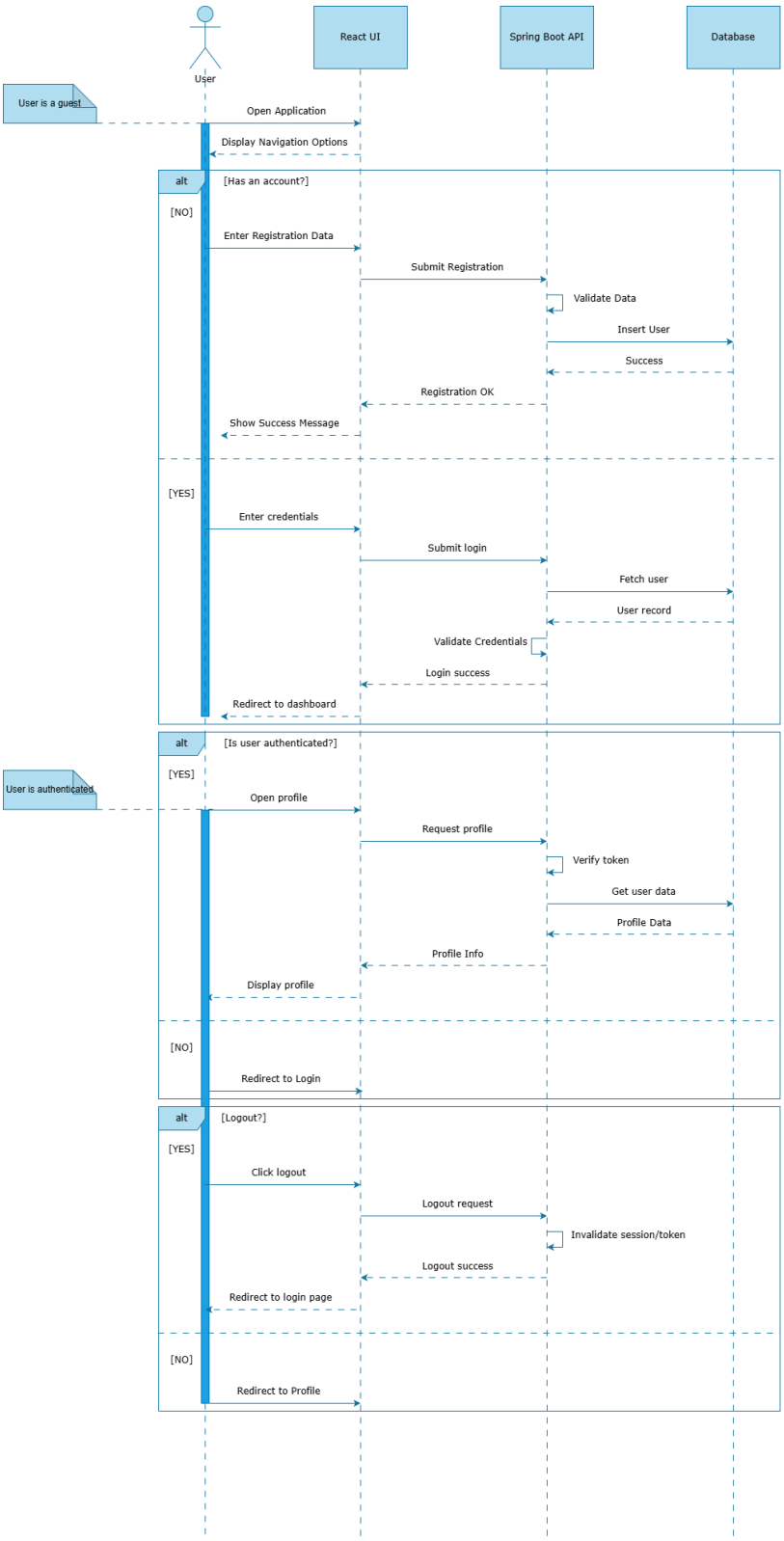
5.4. Class Diagram

Insert ERD here



5.5. Sequence Diagram

Insert ERD here

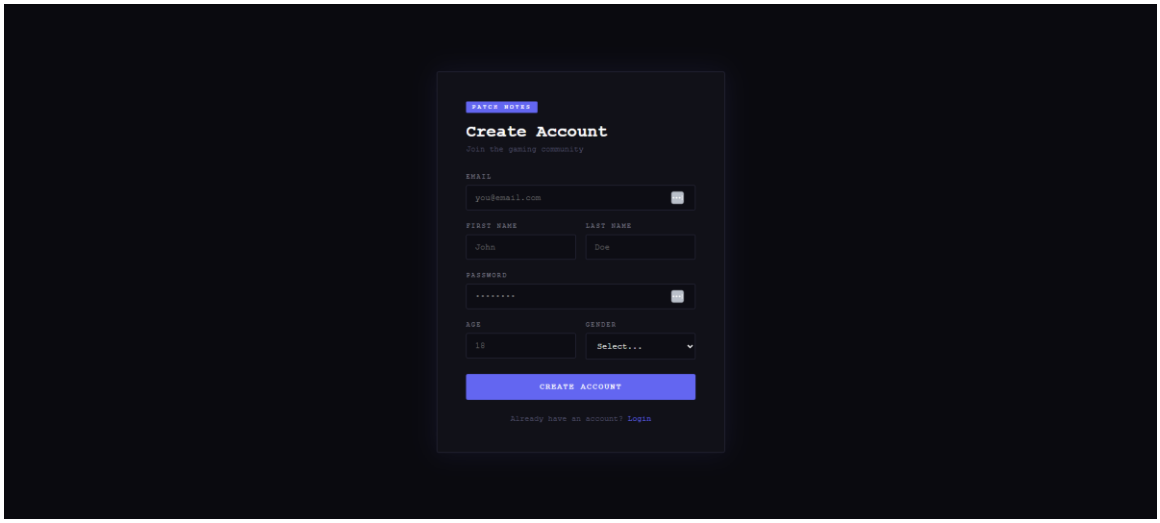


6. Appendices

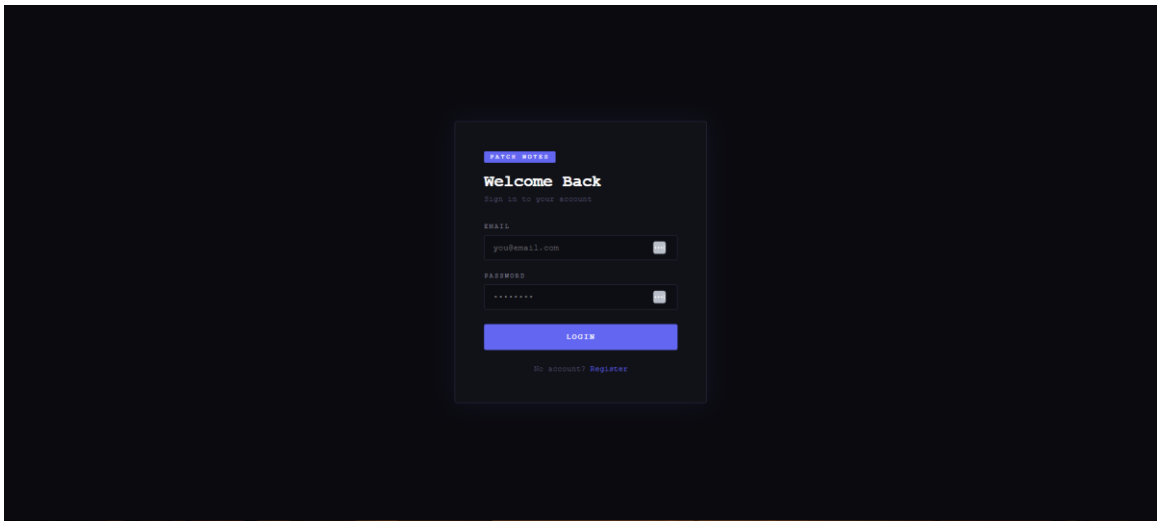
Include any additional information, references, or support materials.

6.1. Screenshots of the Web UI

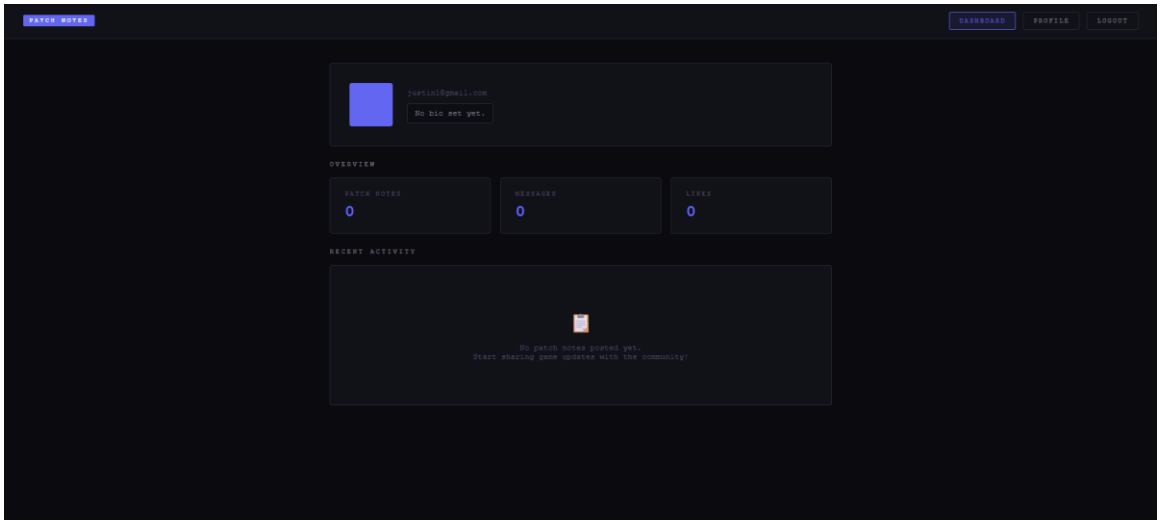
6.1.1. Registration Page



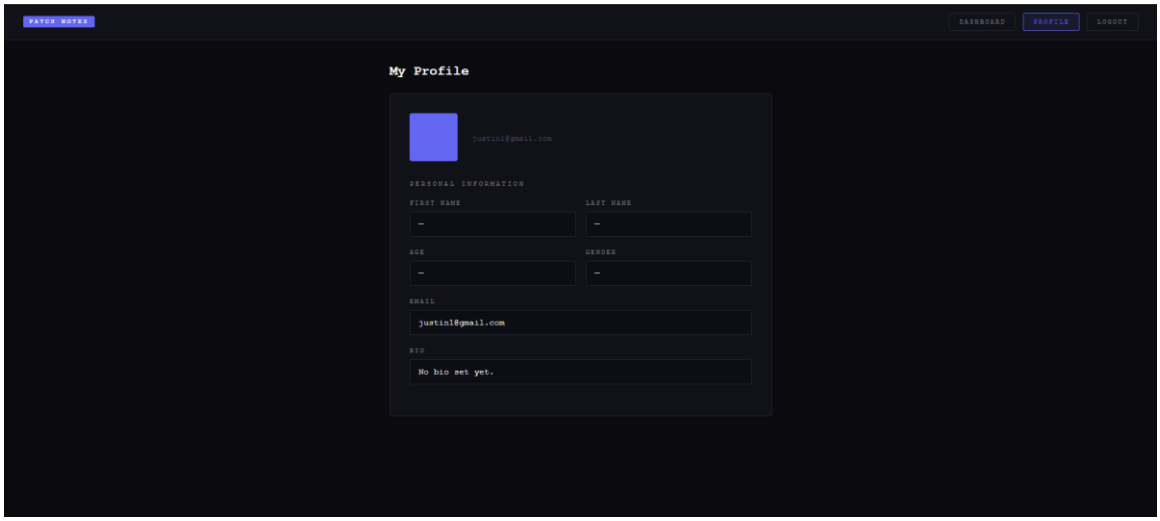
6.1.2. Login Page



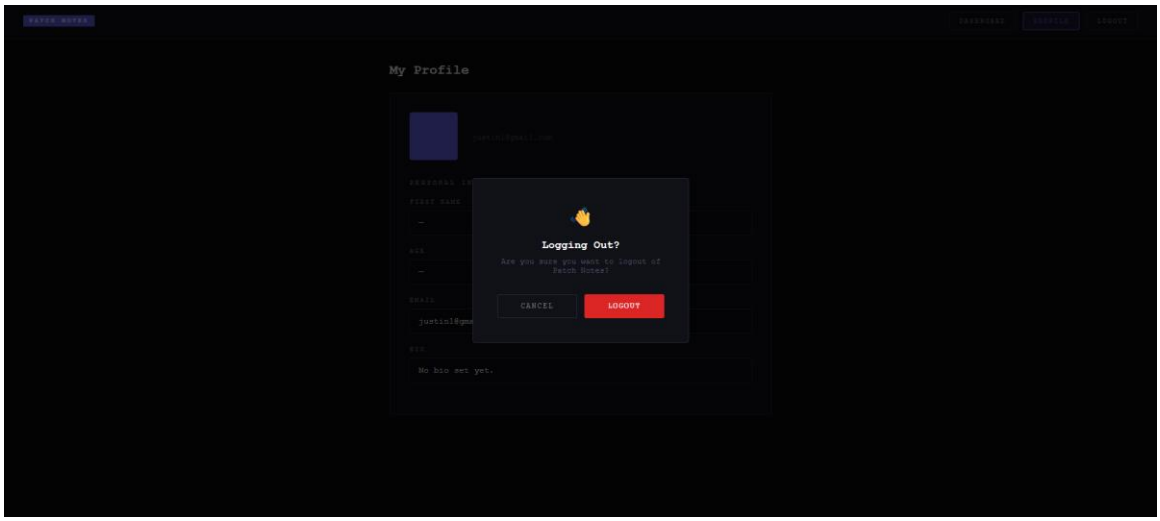
6.1.3. Dashboard/Landing Page



6.1.4. Profile Page

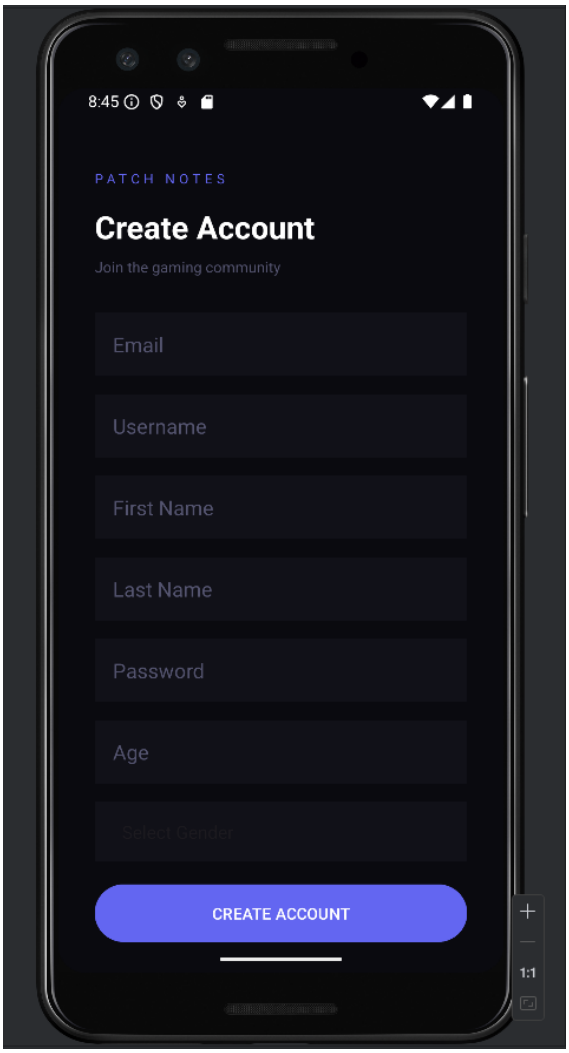


6.1.5. Logout Page

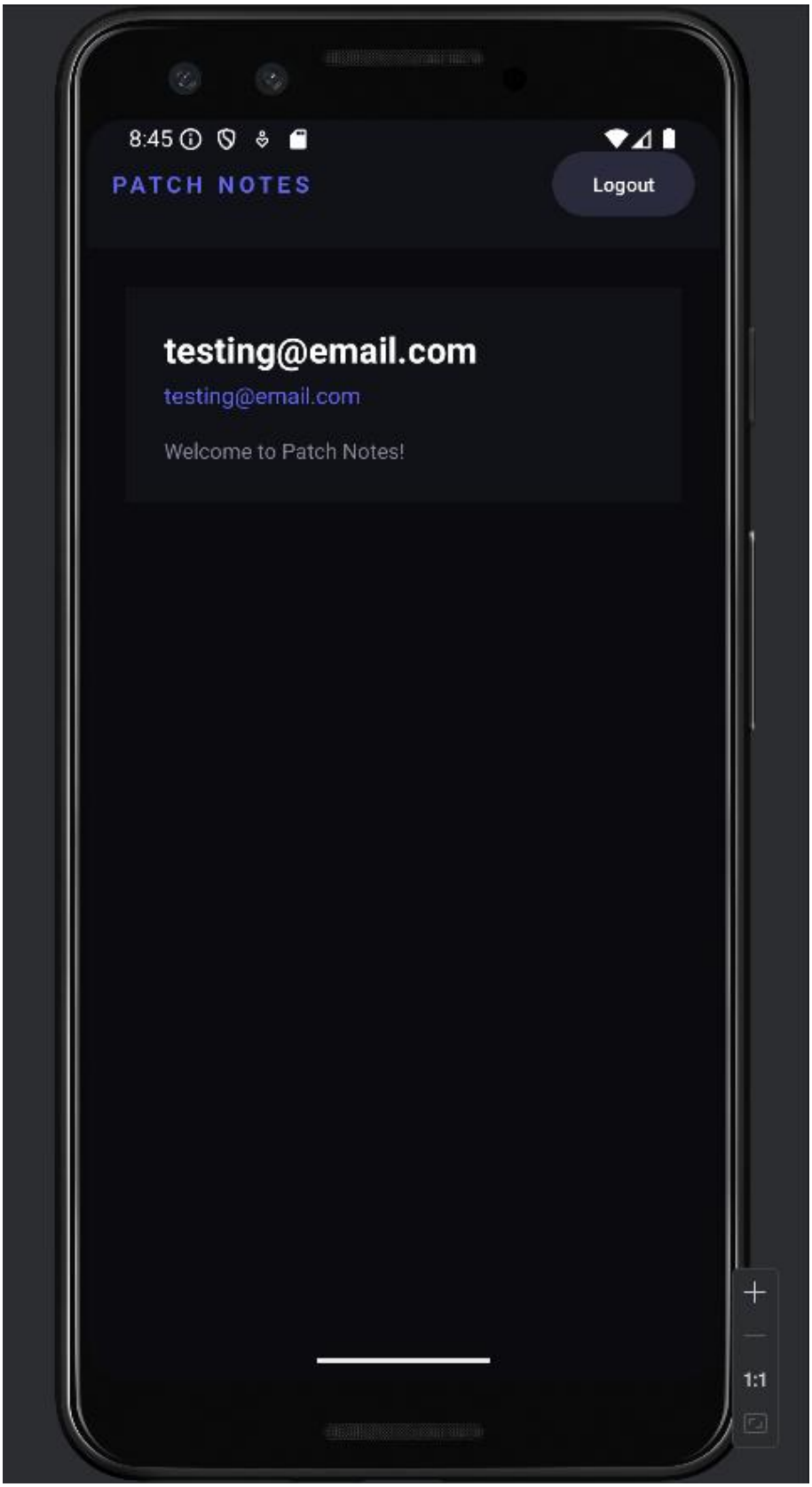


6.2. Screenshots of the Web UI

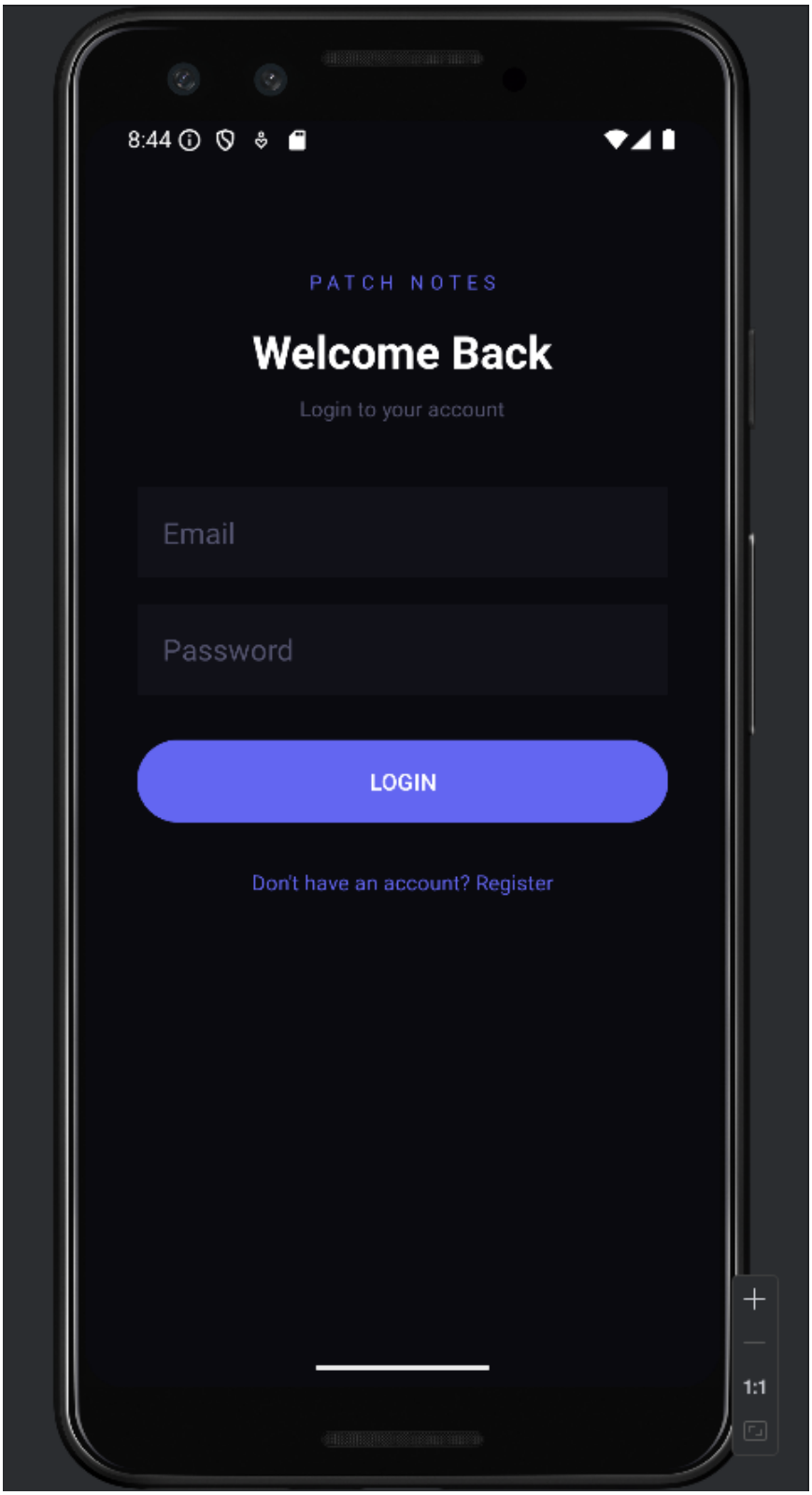
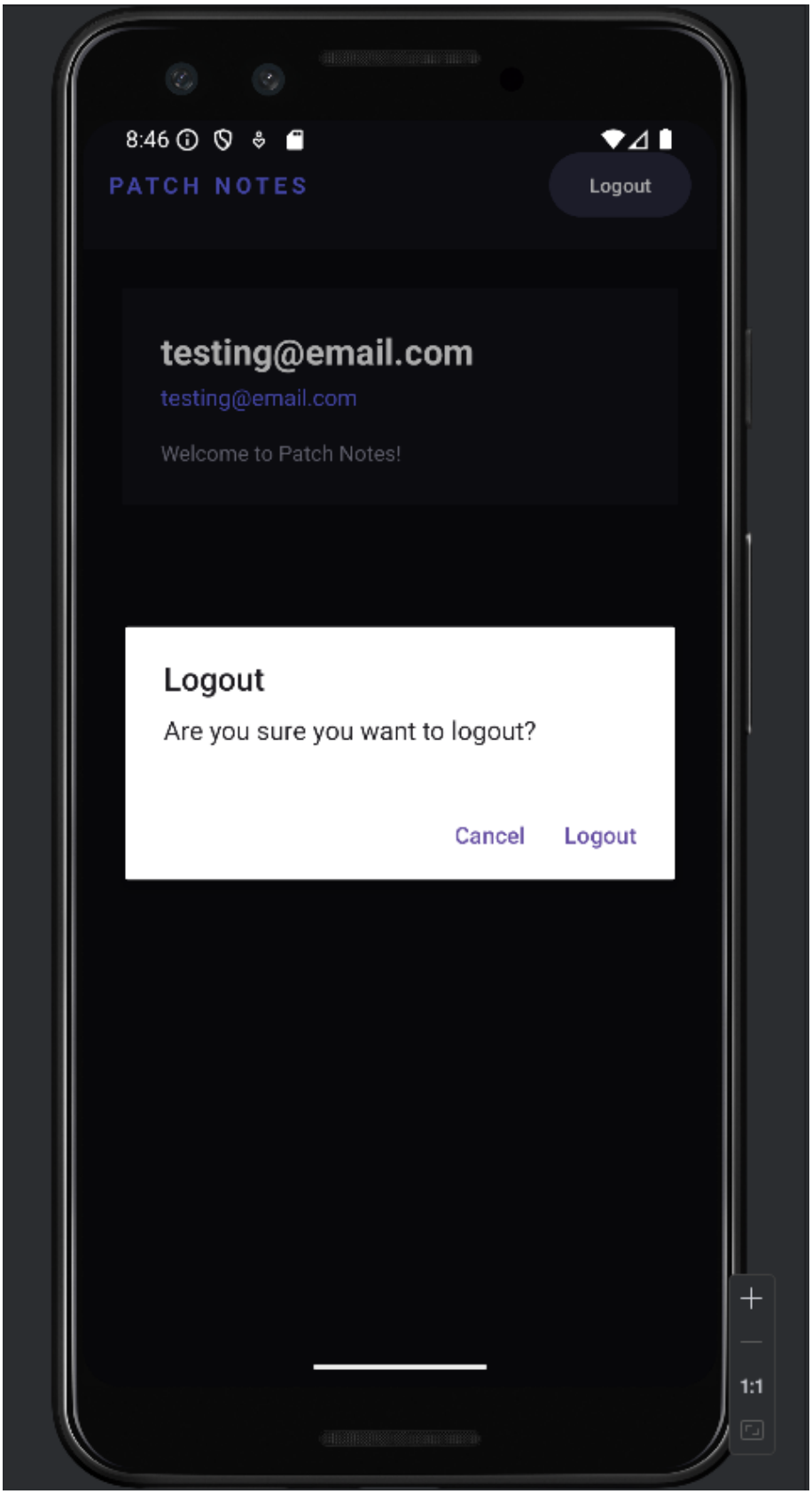
6.2.1. Registration Page



6.2.2. Dashboard Page



6.2.3. Login Page



6.2.4. Logout Page